EduSense AI

Complete Build Plan — AI-Powered Personalized Learning & Student Intelligence SaaS

Purpose: A production-oriented EdTech AI project designed to demonstrate Machine Learning, PyTorch, NLP, recommendation systems, personalization, LLMs, FastAPI, Docker, AWS, and AI product engineering.

How to use this document: Give the entire PDF to Antigravity IDE as the project specification. Build sprint-by-sprint, keep the architecture modular, do not skip tests, and maintain the definition-of-done for every sprint.

1. Product Vision

EduSense AI analyzes student learning behavior and academic performance, predicts areas where a learner may struggle, recommends what to learn next, builds personalized learning paths, analyzes natural-language feedback, and provides an LLM-powered tutor and quiz generator. The product should feel like a real SaaS platform rather than a notebook or demo.

Core learning loop

Student activity → data ingestion → feature engineering → ML prediction → weak-topic detection → recommendation → personalized learning path → LLM tutoring/quiz → new performance data → continuous personalization.

2. Product Goals and Non-Goals

Goals	Non-goals for the first release
Demonstrate end-to-end applied AI engineering	Training a foundation LLM from scratch
Personalized educational recommendations	Complex distributed microservices before needed
Production-style REST APIs and data models	Overbuilding a social/community platform
ML + NLP + LLM integration	Audio/video features
Dockerized, AWS-ready deployment	Enterprise-scale Kubernetes on day one
Strong portfolio/interview evidence	Perfect production SLAs in the first iteration

3. Target Users

• **Student:** views performance, receives recommendations, follows a learning path, asks the AI tutor questions, and takes generated quizzes.

• **Instructor/Admin:** views aggregate learning trends, manages topics/resources, and inspects recommendation and model performance.

• **AI/ML Engineer:** monitors models, data quality, inference behavior, and evaluation metrics.

4. System Architecture

Recommended architecture: Streamlit frontend for the portfolio MVP; FastAPI backend; PostgreSQL for application data; scikit-learn and PyTorch for ML; Hugging Face for NLP; Ollama for local LLM inference; a recommendation service; Docker Compose for local deployment; AWS-ready deployment after stabilization.

```
Frontend (Streamlit) | v FastAPI REST API | +------------------+------------------+------------------+ | | | | v v v v Student/Quiz ML Prediction Recommendation LLM Tutor Service Service Service Service | | | | +------------------+------------------+------------------+ | v PostgreSQL | Learning / Student Data | Training Pipeline | scikit-learn / PyTorch
```

5. Technology Stack

Layer	Technology	Purpose
Language	Python 3.11+	Primary development language
Backend	FastAPI + Pydantic	REST APIs and validation
Frontend	Streamlit	Portfolio MVP dashboard
Database	PostgreSQL	Students, attempts, topics, recommendations
ML	scikit-learn	Baseline and classical ML
Deep Learning	PyTorch	Neural-network implementation
NLP	Hugging Face Transformers	Text classification/embeddings
LLM	Ollama	Local AI tutor and quiz generation
Recommendation	Python + similarity/ML	Content-based personalization
Containerization	Docker + Compose	Reproducible deployment
Cloud	AWS	Deployment and portfolio cloud exposure
Testing	pytest	Unit/API tests
Quality	ruff/black + type hints	Code quality
Version control	Git/GitHub	Source control

6. Recommended Repository Structure

```
edusense-ai/ ■■■ app/ ■ ■■■ api/ ■ ■ ■■■ routes/ ■ ■ ■■■ dependencies.py ■ ■■■ core/ ■ ■ ■■■ config.py ■ ■ ■■■ logging.py ■ ■■■ db/ ■ ■ ■■■ models.py ■ ■ ■■■ session.py ■ ■ ■■■ migrations/ ■ ■■■ schemas/ ■ ■■■ services/ ■ ■ ■■■ student_service.py ■ ■ ■■■ prediction_service.py ■ ■ ■■■ recommendation_service.py ■ ■ ■■■ nlp_service.py ■ ■ ■■■ llm_service.py ■ ■■■ ml/ ■ ■ ■■■ data/ ■ ■ ■■■ features/ ■ ■ ■■■ training/ ■ ■ ■■■ inference/ ■ ■ ■■■ models/ ■ ■■■ recommender/ ■ ■■■ llm/ ■ ■■■ main.py ■■■ frontend/ ■ ■■■ pages/ ■ ■■■ components/ ■■■ data/ ■ ■■■ raw/ ■ ■■■ processed/ ■ ■■■ synthetic/ ■■■ notebooks/ ■■■ tests/ ■■■ scripts/ ■■■ docker/ ■■■ docs/ ■■■ .env.example ■■■ Dockerfile ■■■ docker-compose.yml ■■■ requirements.txt ■■■ README.md ■■■ pyproject.toml
```

7. Data Model

Entity	Important fields
users	id, name, email, role, created_at
student_profiles	user_id, education_level, goals, preferred_difficulty
topics	id, subject, name, difficulty, prerequisites
learning_resources	id, topic_id, type, title, url/path, difficulty
quiz_attempts	student_id, topic_id, score, time_spent, attempts, timestamp
learning_events	student_id, topic_id, event_type, duration, timestamp

Entity	Important fields
student_feedback	student_id, text, topic_id, timestamp
recommendations	student_id, topic_id, score, reason, created_at
learning_paths	student_id, ordered_topics, status, created_at
model_runs	model_name, version, metrics, trained_at

8. ML Design

Primary prediction task: predict whether a student is likely to struggle with a target topic.

Candidate features: recent quiz score, historical topic score, attempts, time spent, completion rate, topic difficulty, prerequisite performance, recent trend, and engagement frequency.

Models: start with Logistic Regression as a baseline, then Random Forest. Add XGBoost only if useful. Later implement a small PyTorch neural network for learning and interview demonstration.

Evaluation: accuracy, precision, recall, F1, ROC-AUC, confusion matrix. Explain why the business may prioritize recall when the cost of missing a struggling student is high.

9. Recommendation Engine

Version 1 should be content-based. Represent topics/resources using metadata and text embeddings or TF-IDF, identify weak topics, filter already-mastered topics, respect prerequisites, score candidates, and return ranked recommendations with an explanation.

```
Student profile + Weak topics + Topic prerequisites + Difficulty preference + Resource metadata | v Candidate generation | v Filtering | v Scoring / similarity | v Top-N recommendations
```

Version 2 may add collaborative filtering using a student-item interaction matrix. Version 3 may combine content and collaborative signals into a hybrid recommender.

10. Personalization Engine

• Adapt difficulty based on recent performance.

• Prioritize prerequisite topics when foundational knowledge is weak.

• Use recent learning behavior more heavily than stale activity.

• Generate an ordered learning path rather than an unordered list.

• Store recommendation reasons so the UI can explain why an item was suggested.

• Continuously update the student profile after quiz attempts and feedback.

11. NLP Module

Build an NLP pipeline for student feedback. Start with preprocessing and TF-IDF/classical classification; then add a Hugging Face transformer for a stronger model. Extract topic/concept signals, classify confusion or sentiment where useful, and feed the results into personalization.

> Learning progression: Bag of Words → TF-IDF → embeddings → transformer → LLM.

12. LLM Tutor

The tutor must use structured student context: current topic, performance, weak areas, preferred difficulty, recent attempts, and the user's question. The system prompt should instruct the model to teach at the student's level, avoid

fabricating educational resources, show reasoning at an appropriate level, and ask a short diagnostic question when the student's level is unclear.

Use Ollama locally for the initial build. Keep the LLM provider behind an interface so another provider can be added later.

13. AI Quiz Generator

Generate multiple-choice and short-answer questions from a topic and difficulty level. Store the generated question, expected answer, explanation, and metadata. Evaluate the student's answer, update performance, and feed the result back into recommendation and personalization.

14. API Contract

Method	Endpoint	Purpose
GET	/health	Health check
POST	/students	Create student profile
GET	/students/{id}	Get student profile
POST	/attempts	Record quiz attempt
GET	/students/{id}/performance	Performance analytics
POST	/predict/struggle	Predict struggle probability
GET	/students/{id}/recommendations	Get ranked recommendations
POST	/learning-path	Generate/update learning path
POST	/feedback/analyze	Analyze feedback with NLP
POST	/tutor/chat	Ask personalized LLM tutor
POST	/quiz/generate	Generate AI quiz
POST	/quiz/evaluate	Evaluate answer
GET	/models	Model/version metadata

15. 12-Sprint Build Roadmap

Sprint 1 — Foundation

Build: Repo, venv, dependency management, FastAPI, Streamlit, PostgreSQL, config, logging, Git, health endpoint.

Definition of done: App boots; DB connects; frontend connects to API; README exists.

Sprint 2 — Data Layer

Build: Synthetic educational dataset, schemas, ingestion, cleaning, EDA, feature engineering, train/validation/test strategy.

Definition of done: Reproducible dataset pipeline and documented features.

Sprint 3 — ML Prediction

Build: Baseline Logistic Regression, Random Forest, evaluation, model persistence, inference service, prediction endpoint.

Definition of done: Model predicts struggle probability through API with metrics.

Sprint 4 — Recommendation Engine

Build: Weak-topic detection, content-based ranking, TF-IDF/embeddings, prerequisites, Top-N recommendations.

Definition of done: Student receives explainable ranked recommendations.

Sprint 5 — Personalization

Build: Student profiles, adaptive difficulty, trend analysis, learning-path generation, recommendation feedback.

Definition of done: Personalized learning path changes with new data.

Sprint 6 — NLP

Build: Feedback preprocessing, baseline classifier, Hugging Face transformer, topic/concept signals.

Definition of done: Feedback analysis updates student intelligence.

Sprint 7 — LLM Tutor

Build: Ollama provider, prompt templates, context builder, safe output handling, conversation history.

Definition of done: Personalized tutor works from API and UI.

Sprint 8 — AI Quiz

Build: Question generation, answer evaluation, difficulty control, attempt storage, performance update.

Definition of done: Closed learning loop works end-to-end.

Sprint 9 — PyTorch

Build: Dataset/DataLoader, nn.Module, training loop, validation, metrics, model export, comparison with classical ML.

Definition of done: Working PyTorch model and documented comparison.

Sprint 10 — Production Backend

Build: Auth, validation, error handling, async where appropriate, structured logging, tests, API docs.

Definition of done: Secure, tested API with useful Swagger docs.

Sprint 11 — Docker

Build: Dockerfile, Compose, environment variables, persistent volumes, health checks.

Definition of done: One-command local deployment.

Sprint 12 — AWS + Portfolio

Build: S3/RDS/EC2 or ECS-ready design, CI/CD basics, monitoring, architecture docs, demo polish.

Definition of done: Deployable portfolio product and interview-ready documentation.

16. Sprint Execution Rules

• Every sprint must end with working code, tests, documentation, and a Git commit.

• Do not put business logic directly in Streamlit pages; keep it in services/backend modules.

• Use Pydantic schemas for API input/output.

• Use environment variables for secrets and model/provider configuration.

• Never hard-code API keys or database passwords.

• Keep ML training separate from inference. Training scripts produce versioned artifacts consumed by inference services.

• Log important inference/model events without logging sensitive student data.

• Use deterministic seeds where appropriate for reproducible experiments.

• Track model version, dataset version, training date, and metrics.

• Write unit tests for recommendation scoring, feature engineering, and service logic.

• Write API tests for critical endpoints.

• Use type hints throughout the codebase.

• Prefer small modules with clear responsibilities over giant files.

• Do not add microservices unless a real boundary requires them.

17. Testing Strategy

• **Unit tests:** feature engineering, risk scoring, recommendation scoring, prerequisite filtering, prompt/context construction.

• **Integration tests:** FastAPI + PostgreSQL, prediction endpoint, recommendation endpoint, quiz flow.

• **Model tests:** schema validation, expected feature columns, model artifact loading, metric thresholds.

• **LLM tests:** prompt contract tests, malformed-output handling, fallback behavior; do not rely only on exact text matches.

• **End-to-end:** create student → record attempts → predict → recommend → generate learning path → ask tutor → generate/evaluate quiz → update profile.

18. Security and Reliability

• JWT-based authentication can be added in Sprint 10.

• Students must only access their own data.

• Validate all API inputs.

• Never expose internal model paths, credentials, or stack traces to clients.

• Use rate limiting or request controls when deploying LLM endpoints.

• Set timeouts around external/model calls.

• Handle unavailable LLM/model services gracefully.

• Use database constraints and transactions for important updates.

• Keep secrets in .env locally and AWS Secrets Manager/Parameter Store in cloud deployments.

19. Docker and AWS Target Architecture

```
Local: Docker Compose ■■■ FastAPI ■■■ Streamlit ■■■ PostgreSQL AWS-ready: User ↓ Load Balancer / reverse proxy ↓ Containerized FastAPI ■■■ PostgreSQL / RDS ■■■ S3 for datasets/artifacts ■■■ ECR for images ■■■ CloudWatch for logs/metrics ■■■ LLM provider / model service
```

20. CI/CD Target

Use GitHub Actions to run linting, tests, build the Docker image, and optionally push the image to ECR. Deployment can be manual initially and automated after the application is stable.

21. Observability

• API latency and error rate.

• Prediction endpoint latency and model version.

• Recommendation generation latency.

• LLM latency and failure rate.

• Number of successful/failed quiz generations.

• Data-quality checks during training.

• Model metrics across versions.

22. ML Evaluation and Business Metrics

Technical metrics alone are insufficient. Track whether recommendations are useful. Candidate product metrics include recommendation click/acceptance rate, quiz completion, learning-path completion, score improvement, and repeat engagement. For the prediction model, track F1/recall and calibration as appropriate.

23. Interview Talking Points This Project Must Demonstrate

• Why did you choose Random Forest over Logistic Regression?

• How did you prevent data leakage?

• How did you split time-dependent learning data?

• Why is recall important for identifying struggling students?

• How does content-based recommendation work?

• How would you add collaborative filtering?

• How would you evaluate a recommendation system?

• Why use Hugging Face instead of an LLM for every NLP task?

• Why use an LLM for tutoring rather than a classical model?

• How do you personalize an LLM response?

• How would you reduce hallucinations?

• How would you scale the FastAPI service?

• How would you deploy this on AWS?

• How would you monitor model drift?

• How would you protect student data?

• What would you change at 100,000 users?

24. Portfolio Definition of Done

• Clean GitHub repository with meaningful commit history.

• README containing product vision, architecture, setup, API examples, model results, and screenshots.

• Architecture diagram.

• Data dictionary and model-card style documentation.

• Working FastAPI backend.

• Working Streamlit dashboard.

• At least one classical ML model and one PyTorch model.

• Working recommendation engine.

• Working NLP module.

• Working local LLM tutor.

• Working AI quiz generator.

• Docker Compose deployment.

• Automated tests and CI checks.

• AWS deployment plan or actual deployment.

• Short demo video/GIF if possible.

25. Antigravity IDE Master Build Prompt

Use the following as the master instruction when initializing the repository. Antigravity should treat the PDF as the source of truth and execute one sprint at a time.

```
You are the lead AI engineer building EduSense AI, a production-oriented SaaS prototype for personalized education. Read the complete project specification before changing code. Build the system sprint-by-sprint in the order defined in the specification. Do not skip architecture, testing, documentation, validation, or security requirements. Primary stack: Python 3.11+, FastAPI, Pydantic, Streamlit, PostgreSQL, scikit-learn, PyTorch, Hugging Face Transformers, Ollama, Docker, pytest, GitHub Actions, AWS-ready architecture. Engineering rules: 1. Keep frontend, API, services, ML training, ML inference, recommendation, NLP, and LLM code separated. 2. Use environment variables for configuration and secrets. 3. Use typed Python code and clear module boundaries. 4. Do not hard-code secrets. 5. Training and inference must be separate. 6. Save model metadata and metrics. 7. Add tests with every meaningful feature. 8. Keep API contracts documented. 9. Make local development reproducible. 10. Prefer simple, maintainable architecture over unnecessary microservices. 11. Never fabricate completed functionality. Mark incomplete features explicitly. 12. After each sprint, run tests, verify the application, update README/docs, and report: files changed, commands run, tests passed, known issues, and next sprint. 13. Do not silently redesign the product. If a design change is necessary, explain the trade-off first. 14. The final system must demonstrate ML, recommendation, personalization, NLP, LLM integration, REST APIs, Docker, and AWS readiness. Start with Sprint 1 only. Do not implement later sprints until Sprint 1 is verified.
```

26. Final Product Summary

> EduSense AI should ultimately demonstrate that the builder can take raw educational data, transform it into useful features, train and evaluate ML models, build recommendation and personalization logic, integrate NLP and LLM capabilities, expose the system through production-style APIs, test and containerize it, and prepare it for cloud deployment.

Strategic portfolio pairing: Athena AI demonstrates enterprise GenAI, RAG, agents, memory, multimodal AI, streaming, authentication, and SaaS architecture. EduSense AI demonstrates applied ML, recommendation systems, personalization, NLP, PyTorch, EdTech intelligence, and production ML deployment. Together they provide a much broader AI Engineer portfolio aligned to the target job.